

Serving Long-Context LLMs at the Mobile Edge: Test-Time Reinforcement Learning-Based Model Caching and Inference Offloading

Minrui Xu^{1b}, *Member, IEEE*, Dusit Niyato^{2b}, *Fellow, IEEE*, and Christopher G. Brinton^{3b}, *Senior Member, IEEE*

Abstract—Large Language Models (LLMs) can perform zero-shot learning on unseen tasks and few-shot learning on complex reasoning tasks. However, resource-limited mobile edge networks struggle to support long-context LLM serving for LLM agents during multi-round interactions with users. Unlike stateless computation offloading and static service offloading in edge computing, optimizing LLM serving at edge servers is challenging because LLMs continuously learn from context which raises accuracy, latency, and resource consumption dynamics. In this paper, we propose a joint model caching and inference offloading framework that utilizes test-time deep reinforcement learning (T2DRL) to optimize deployment and execution strategies for long-context LLM serving. In this framework, we analyze the performance convergence and design an optimization problem considering the utilization of context windows in LLMs. Furthermore, the T2DRL algorithm can learn in both the training phase and the testing phase to proactively manage cached models and service requests and adapt to context changes and usage patterns during execution. To further enhance resource allocation efficiency, we propose a double Dutch auction (DDA) mechanism, which dynamically aligns the marginal value of an additional reasoning path with the marginal cost of reasoning services. Finally, experimental results demonstrate that the T2DRL algorithm can reduce system costs by at least 30% compared to baselines while guaranteeing the performance of LLM agents in real-world perception and reasoning tasks.

Index Terms—Mobile edge networks, large language models (LLMs), deep reinforcement learning (DRL), auction theory.

I. INTRODUCTION

LARGE Language Models (LLMs) [1] have billions of parameters and demonstrate emerging capability through

Received 15 January 2025; revised 22 September 2025 and 25 January 2026; accepted 20 February 2026; approved by IEEE TRANSACTIONS ON NETWORKING Editor J. Liu. Date of publication 27 February 2026; date of current version 6 March 2026. This work was supported in part by the Seatrium New Energy Laboratory, Singapore Ministry of Education (MOE) Tier 1 under Grant RT5/23 and Grant RG24/24; in part by Nanyang Technological University (NTU) Centre for Computational Technologies in Finance (NTU-CCTF); and in part by the Research Innovation and Enterprise (RIE) 2025 Industry Alignment Fund-Industry Collaboration Projects (IAF-ICP), administered by the Agency for Science, Technology and Research (A*STAR) under Award I2301E0026. The work of Christopher G. Brinton was supported in part by the National Science Foundation (NSF) under Grant CPS-2313109, in part by the Office of Naval Research (ONR) under Grant N00014-22-1-2305, and in part by the Air Force Office of Scientific Research (AFOSR) under Grant FA9550-24-1-0083. (*Corresponding author: Minrui Xu.*)

Minrui Xu and Dusit Niyato are with the College of Computing and Data Science, Nanyang Technological University, Singapore 639798 (e-mail: minrui001@e.ntu.edu.sg; dniyato@ntu.edu.sg).

Christopher G. Brinton is with the Elmore Family School of Electrical and Computer Engineering, Purdue University, West Lafayette, IN 47906 USA (e-mail: cgb@purdue.edu).

Digital Object Identifier 10.1109/TON.2026.3669011

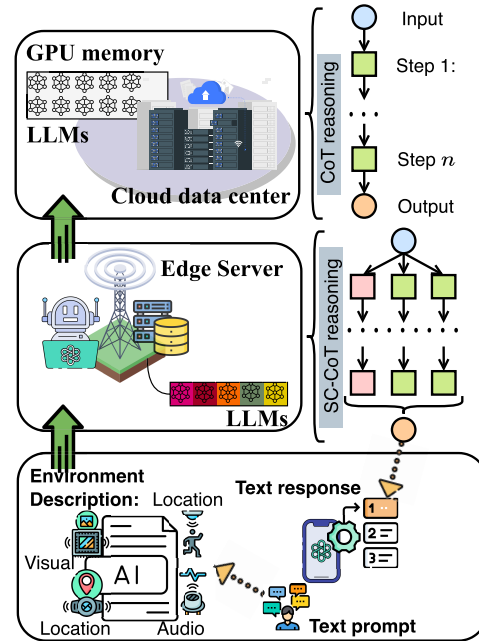


Fig. 1. Serving LLMs to handle inputs and tackle complex tasks using CoT prompting in mobile edge networks.

pre-training on internet-scale datasets. In mobile edge networks, LLMs can empower mobile devices to perceive environments and plan for future actions, for running LLM agents on devices and enabling zero-shot learning for unseen perception tasks and few-shot learning for complex reasoning tasks [2], [3]. For instance, LLMs can empower mobile devices with environmental understanding and reasoning capabilities, allowing them to adapt flexibly to changing conditions [4]. Furthermore, LLMs improve the experience of user interaction and achieve automation for mobile devices during the inference process by leveraging the contextual data derived from environmental perceptions and user interactions. However, optimizing long-context LLM serving in resource-constrained edge networks remains an open challenge [5] because existing offloading frameworks in mobile edge networks cannot accommodate LLMs' in-context learning capability.

As illustrated in Fig. 1, resource-intensive tasks such as LLM serving, i.e., the deployment and inference of LLMs to perform inference tasks, in mobile edge networks [5], often need to be offloaded to edge servers (ESs) or cloud data centers to reduce latency and improve resource utilization

efficiency [6]. Due to the unique context-aware characteristics of LLMs that they can learn in-context examples during few-shot learning [2], LLM serving at ESs is significantly different from computation and service offloading [7], [8]. Specifically, computation offloading involves transferring computation-intensive tasks from local devices to ESs or a cloud data center [9], which is stateless and independent of previous computation outcomes or the historical results of the servers. Similarly, service offloading involves sending user inputs to ESs to invoke pre-cached services [10], which is static and remains unchanged across different invocations. As a result, existing optimization frameworks for computation offloading and service offloading are no longer effective for optimizing long-context LLM serving, as they fail to accommodate the dynamic learning capabilities of LLMs and the need for service adaptation based on evolving model contexts.

Similar to content and service caching, model caching for LLMs is an emerging solution for provisioning LLM agents at ESs [11], [12]. Specifically, LLMs can learn from input examples and adjust the output by eliciting their reasoning capability via prompting techniques, such as Chain-of-Thought (CoT) [13] and Self-Consistent CoT [14] prompting. These methods improve accuracy, but they also increase resource consumption and latency. The performance of an LLM agent is limited not only by the hardware resources available at edge servers but also by the model's context window, which is determined by the architecture. If the generated tokens exceed this window, the cached model must be evicted; otherwise, performance declines sharply. To optimize long-context LLM serving in mobile edge networks, cached models and inference requests must be actively managed in response to the state transitions of requests and cached models.

In this paper, we propose a joint model caching and inference offloading framework for long-context LLM serving in mobile edge networks, which allows LLMs to adopt various prompting techniques. In this framework, we formulate the problem of long-context LLM serving as a mixed-integer programming problem to minimize the system cost under the hardware resource constraints and context window constraints. To evaluate the performance of LLMs theoretically, we analyze the convergence rate of language ambiguity with respect to the reasoning paths and steps. To obtain optimal cache and inference management strategies, we propose a test-time deep reinforcement learning (T2DRL) algorithm [15], [16] utilizing test-time training (TTT) models as the actor-critic network that can learn not only from historical experience during training but also from real-time interaction during testing. By treating the hidden state as a learnable model, the TTT model-based policy can be updated through self-supervised learning during test time, which overcomes the limited expressive power in Long Short-Term Memory (LSTM) networks and the quadratic complexity of attention mechanisms. Building on the T2DRL optimization results, we cast long-context LLM serving as a two-sided market and employ a Double Dutch Auction (DDA) to allocate edge compute and cache efficiently. The mechanism runs synchronized buyer and seller clocks that post a common price; requests claim units while their estimated marginal accuracy gain exceeds the price, and servers release

capacity when their marginal cost is covered. This procedure aligns supply with demand in real time, clears at a uniform price, and reduces strategic overhead, which yields low-latency and cost-effective allocations. It scales across heterogeneous edge servers and directs additional compute to prompts that benefit most from multi-path reasoning, thereby improving social welfare while preserving transparency.

The main contributions can be summarized as follows.

- For long-context LLM serving in mobile edge networks, we propose a joint model caching and inference offloading framework that adapts to dynamic network conditions and device capabilities.
- In the framework, we analyze the convergence rate of LLMs handling contexts with CoT prompting. This approach enhances the reasoning and planning capability of LLMs by generating intermediate reasoning thoughts with additional latency and resources.
- Furthermore, we model a long-term optimization problem to minimize total system costs in terms of edge inference cost at ESs and the cloud inference cost of the cloud data center. We formulate this long-context LLM serving problem as a Markov decision process (MDP) where model caching and inference offloading decisions can affect the transition of states.
- We introduce a Double Dutch Auction mechanism to allocate LLM services with a uniform price per batch, supports multi-unit adaptation within the latency budget, and complements test-time scaling by allocating more paths exactly when the observed self-consistency signals indicate larger accuracy gains.
- To achieve a proactive cached model and inference management, we propose a test-time deep reinforcement learning algorithm that learns optimal model caching strategies with a test-time training model-based actor-critic network. This algorithm can adjust to the dynamic requests and contexts during the testing phase, balancing performance with computational efficiency.

The remainder of this paper is organized as follows. Section II reviews related work on serving LLMs in mobile edge networks, caching strategies, and reinforcement learning approaches. Section III introduces the system model and problem formulation. Section IV details the proposed T2DRL algorithm and its optimization process. Section V formalizes this market and describes the auction dynamics and guarantees. Section VI presents experimental results and performance evaluations. Finally, Section VII concludes the paper and outlines future research directions.

II. RELATED WORK

A. Serving Large Language Models in Mobile Edge Networks

In mobile edge networks, there are three main objectives for serving LLMs, including context, accuracy, and performance [5], [9]. Context is the ability of LLMs to handle longer contextual inputs effectively, which can improve their ability to understand and generate responses based on more extensive and potentially more complex histories during inference. The

TABLE I
COMPARISON AGAINST PRIOR WORK WITH A CONCISE ALLOCATION MECHANISM SUMMARY

Test-time adaptation		
Method	Test-time DRL	Generic test-time adaptation
Objective	Minimize serving cost by combining accuracy, latency, switching, and compute	Reduce predictive loss on test data
What adapts	Policy over caching and offloading	Model or hidden state parameters
Output	Dynamic actions for each request step	Static parameters based on statistics
State	Request mix, cache state, age of thought, and other serving signals	Input features and activation statistics
Resource awareness	Explicit constraints and costs	Usually indirect or absent
Allocation mechanism		
Method	Double Dutch auction	Iterative double auction
Price discovery	Synchronized buyer and seller clocks; agents stop at value or cost; aggregates demand and supply	One side updates per round; sequential bid ask updates
Clearing rule	Uniform clearing price from terminal clocks; optional weighted last clock; strong budget balance	Clears when spread closes or by rule; prices vary by round or pair
Incentives and rationality	Truthful stopping is weakly dominant; individually rational; near competitive equilibrium	Truthfulness not guaranteed; properties depend on protocol
Multi-unit and dynamics	Incremental quantities; stop when marginal value falls below price; adapts within a round	Sequential quantity updates; limited intra-round adaptation
Operational latency	Low overhead via tunable step size; frequent clears reduce queueing.	May require many rounds to clear
Fit to Test-time Scaling	Buyer value ties to marginal accuracy per path; seller cost reflects compute and cache.	Not tailored to marginal accuracy pricing; extra logic for CoT

inference performance can be measured by inference speed, throughput (e.g., tokens per second), and cost efficiency (e.g., cost per token). To balance the accuracy and performance of LLM serving in 6G, Lin et al. in [17] propose a collaborative framework for efficient edge training and inference, leveraging advanced techniques like split learning and quantization to achieve low-latency inference despite limited edge computing resources. Furthermore, a personalized inference scheduling framework is proposed in [18], which formulates LLM serving as a combinatorial multi-armed bandit problem and proposes a constraint satisfaction upper confidence bound (UCB) algorithm for inference scheduling and resource allocation. To preserve data privacy during LLM serving, Su et al. in [19] present a novel distributed training paradigm designed specifically for fine-tuning pre-trained LLMs in a privacy-preserving manner directly on client devices. From the perspective of resource utilization efficiency, Ye et al. in [6] propose Galaxy by incorporating a heterogeneity and memory-budget aware workload planning algorithm, and Yuan et al. in [20] use the UCB algorithm to determine energy-efficient configurations.

B. Service Caching and Offloading in Mobile Edge Networks

Service caching and offloading allow for the full utilization of heterogeneous edge resource capacities for executing computation-intensive services in mobile edge networks. To minimize service provisioning delay, Zhou et al. in [21] leverage the Lyapunov optimization approach and dependent rounding technique to determine caching and offloading decisions under convergence constraints. Considering cooperative service caching and workload scheduling, Ma et al. in [10] propose an iterative algorithm based on Gibbs sampling and a heuristic workload scheduling algorithm with polynomial complexity. To enhance the efficiency of Federated

Learning (FL), a framework of cache-based FL model training is developed in [8] for reducing per-iteration training time. Furthermore, edge caching for model downloading is investigated in [22]. During LLM serving, the concept of context caching is proposed in [11] that adopts a paging-based dynamic memory management system and explores efficient Key-Value (KV) cache transfer methods.

C. Deep Reinforcement Learning for Edge Caching

DRL can learn proactive caching decisions to enhance cache hit rates by allowing the system to predict future content requests and cache popular content in dynamic edge systems [23]. To handle the temporal-spatial dynamic cache popularity, LSTMs and attention mechanisms are leveraged to improve the performance of DRL. For instance, LSTM-based DRL can dynamically adapt to changing user behaviors and environmental conditions, making them more effective than static caching strategies [24], [25]. Moreover, attention-based DRL can capture the influence of adjacent ESs on the current node, which helps in making more informed caching decisions, which leads to improved caching efficiency and reduced access to the core network [26], [27]. Considering the increasing popularity and demand for LLM applications on mobile devices, He et al. in [28] propose an active inference approach for offloading and resource allocation of LLM inference tasks in the edge-cloud collaboration framework, which dynamically adapts to varying task loads and resource constraints in mobile edge networks. Nevertheless, during long-context inference, LSTMs have limited expressive power in their hidden states while attention mechanisms have a quadratic complexity [16]. Therefore, in this paper, we proposed a novel T2DRL algorithm with linear complexity and an expressive hidden state to handle long contexts efficiently. As listed in TABLE I, generic

TABLE II
MAIN NOTATIONS IN THE PAPER

Symbol	Description
$\mathcal{N}$	Set of LLM agent providers; 0 denotes the cloud data center and $n \in \mathcal{N}$ denotes an ES.
$\mathcal{I}$	Set of LLM agents; $i \in \mathcal{I}$.
$\mathcal{M}$	Set of LLMs; $m \in \mathcal{M}$.
t	Discrete time slot.
U_n	User group attached to ES n .
$R_{n,i,m}^t$	Requests from U_n that require model m for agent i in slot t [requests per slot].
d_i	Input size for agent i [tokens].
s_m	GPU memory footprint of model m [GB].
e_m	Energy per token for model m .
w_m	Context window size of model m [tokens].
G_n	GPU memory capacity at ES n [GB].
E_n	Energy capacity at ES n .
f_n	Computing capacity of ES n .
$a_{n,i,m}^t$	Caching indicator for model m used by agent i at ES n in slot t ; $a_{n,i,m}^t \in \{0, 1\}$.
$b_{n,i,m}^t$	Fraction of (n, i, m) requests executed locally at ES n in slot t ; $b_{n,i,m}^t \in [0, 1]$.
$\mathbf{s}_t$	System state at time t .
$\mathbf{a}_t$	Joint action at time t .
r_t	Reward at time t defined as the negative of the total system cost.
π_n	Control policy for ES n .
c^*	True context in the ambiguity analysis.
θ^*	True intention in the ambiguity analysis.
E_i	Chain-of-thought example for instance i .
$\epsilon(E_i)$	Ambiguity measure of E_i with $\epsilon(E_i) \in [0, \sigma]$, $\sigma \in (0, \frac{1}{2}]$.
$\hat{q}(\cdot)$	Model-induced probability measure used in the analysis.
$J_{i,m}$	Number of SC-CoT reasoning paths sampled for instance i with model m .
ρ_j	The j th reasoning path.
o_j	Output produced by path j .
o^*	Final answer selected by majority vote over $\{o_j\}_{j=1}^{J_{i,m}}$.
$\mathbb{I}[\cdot]$	Indicator function.
$K_{n,i,m}^t$	Accumulated number of thoughts for (n, i, m) at time t .
$\kappa_{n,i,m}^t$	Age of thoughts for (n, i, m) at time t .
ζ_i	Consensus factor for SC-CoT reasoning of agent i .
$\Delta_{i,m}^t$	Vanishing factor of thoughts for (i, m) at time t .
$\alpha_{i,m}$	Zero-shot accuracy of model m for agent i .
β	Reasoning gain factor for self-consistency.
$A_{n,i,m}^t$	Accuracy term used in the performance model.
L_n^t	Edge inference cost of ES n in slot t .
L_C^t	Cloud inference cost in slot t .
L_n^{total}	Time-averaged total cost for ES n .

test-time adaptation methods [16] adapt model or hidden state parameters to reduce a predictive loss. T2DRL instead treats test time operation as a sequential control problem. The policy observes request mix and cache state, then issues caching and offloading actions that minimize a composite serving cost, capturing accuracy, latency, switching, and compute usage. This moves adaptation from per-sample loss reduction to end-to-end serving optimization under explicit resource constraints. Furthermore, we employ a Double Dutch Auction [29], [30] to allocate test time computing. The mechanism runs synchro-

nized buyer and seller clocks, accepts stops when price meets private value or cost, and clears trades at a uniform market price determined by the terminal clocks, with a weighted last clock variant for the marginal trade. The mechanism is strongly budget balanced, encourages truthful stopping and guarantees individual rationality, supports multi-unit demand with incremental commitment, and can be optimized for low-latency clearing. In the SC-CoT setting, buyer value equals the marginal accuracy gain of an added reasoning path, and seller cost reflects resources, so the DDA allocates units to the highest marginal value first and incentivizes LLMs to think.

III. SYSTEM MODEL

In mobile edge networks, users can access LLM agents to answer questions or handle complex tasks autonomously for them. Each LLM agent is empowered by one or multiple LLMs. Every user group engages a single LLM agent as their active assistant based on their specific needs and tasks. The network comprises $N + 1$ LLM agent providers, including N ESs equipped with computing units and a centralized cloud data center. The set of LLM agent providers is denoted by $\mathcal{N} = \{0, 1, \dots, N\}$ and the cloud data center by 0. These ESs can execute LLM agents, while additional inference requests can be offloaded to the cloud data center. The set of available LLM agents is represented as $\mathcal{I} = \{1, 2, \dots, I\}$, which relies on one or multiple LLMs whose set is denoted by $\mathcal{M} = \{1, \dots, M\}$. Given the multi-functionality of LLMs in providing various LLM agents, it is typically observed that the number of LLM agents, I , is significantly more than the number of LLMs, i.e., $I \gg M$. For each user group $\mathcal{U}_n$ associated with ES n , the demand for LLM agents is represented by $R_n^t = \{R_{n,i,m}^t | i \in \mathcal{I}, m \in \mathcal{M}\}$, which presents the number of requests for LLM m to perform specific tasks. The input data size for LLM agent i is denoted by d_i , and the configuration of each LLM m includes model size s_m , computation required per token e_m , and context window size w_m . The context window size is a unique characteristic of LLMs, which refers to the maximum number of tokens that an LLM can effectively process and utilize in long-term inference tasks. The main notations used in this paper are listed in TABLE II.

A. Network Model

To optimize the serving of LLMs in mobile edge networks, we propose a joint model caching and inference framework that enables ESs to deploy LLMs for local execution and offload inference requests to cloud data centers for remote execution. In the framework, ESs can determine the cached models and offloaded requests based on their local caching and offloading strategies. Here, the binary variable $a_{n,i,m}^t \in \{0, 1\}$ specifies if LLM m for agent i is cached at ES n during time slot t , and the continuous variable $b_{n,i,m}^t \in [0, 1]$ indicates the fraction of inference requests i for LLM m being executed at ES n at that time slot. The set $\mathbf{a}_n^t = \{a_{n,1,1}^t, \dots, a_{n,I,M}^t\}$ encapsulates the model caching decisions at ES n . Similarly, the inference offloading decision for ES n is captured by $\mathbf{b}_n^t = \{b_{n,1,1}^t, \dots, b_{n,I,M}^t\}$. Storage, switching, and compute costs scale with the model's weight footprint $s_m(q_m)$. To capture

TABLE III
FEW-SHOT PERFORMANCE OF LAMDA-137B ON VARIOUS REASONING TASKS (ACCURACY (%))

Paths	ASDiv		MultiArith		SVAMP		GSM8K		Commonsense QA		Strategy QA		ARC	
	CoT	SC-CoT	CoT	SC-CoT	CoT	SC-CoT	CoT	SC-CoT	CoT	SC-CoT	CoT	SC-CoT	CoT	SC-CoT
0	49	44	52	47	39	34	17	15	58	55	65	63	55	51
5	49	53	52	63	39	43	17	21	58	61	65	66	55	57
10	49	56	52	69	39	49	17	24	58	62	65	67	55	58
15	49	57	52	71	39	51	17	25	58	62	65	67	55	59
20	49	58	52	73	39	52	17	26	58	62	65	68	55	59

TABLE IV
FEW-SHOT PERFORMANCE OF PALM-540B ON VARIOUS REASONING TASKS (ACCURACY (%))

Paths	ASDiv		MultiArith		SVAMP		GSM8K		Commonsense QA		Strategy QA		ARC	
	CoT	SC-CoT	CoT	SC-CoT	CoT	SC-CoT	CoT	SC-CoT	CoT	SC-CoT	CoT	SC-CoT	CoT	SC-CoT
0	74	71	94	89	79	71	56	50	79	75	75	74	85	78
5	74	78	94	97	79	83	56	65	79	79	75	78	85	86
10	74	80	94	98	79	85	56	70	79	80	75	79	85	87
15	74	81	94	98	79	85	56	71	79	80	75	80	85	88
20	74	81	94	98	79	85	56	72	79	80	75	80	85	88

cloud heterogeneity, we additionally index offloading cost by GPU type $g \in G$ with distinct price and performance, and we account for cold starts and egress. Switching cost is proportional to the bytes moved over node I/O paths rather than a constant penalty [9]. These refinements retain the original learning formulation while improving cost realism.

When ESs are constrained by local resources, user-requested LLM agents can be executed locally if the necessary models are already cached in the GPUs. Let G_n represent the GPU memory capacity at ES n . The decisions regarding which models to cache, denoted as $\mathbf{a}_n^t$, must adhere to the GPU memory limits for each time t for every ES, described by:

$$\sum_{i \in \mathcal{I}} \sum_{m \in \mathcal{M}} a_{n,i,m}^t s_m \leq G_n. \quad (1)$$

This constraint imposes that ESs cannot accommodate all LLMs on their GPUs simultaneously. When the LLMs are cached in GPUs, the LLM agents can be served by the ESs. The local inference execution of LLMs at ES n is further limited by

$$b_{n,i,m}^t \mathbb{I}(R_{n,i,m}^t > 0) \leq a_{n,i,m}^t, \forall i \in \mathcal{I}, m \in \mathcal{M}, \quad (2)$$

where $\mathbb{I}(\cdot)$ is the identity function, and $\mathbb{I}(R_{n,i,m}^t > 0)$ indicates that there are active requests for agent i for LLM m at ES n .

Furthermore, the computing resources for executing LLMs are limited by the energy capacity E_n at ES n , which can be represented as

$$\sum_{i \in \mathcal{I}} \sum_{m \in \mathcal{M}} e_m a_{n,i,m}^t b_{n,i,m}^t R_{n,i,m}^t \leq E_n. \quad (3)$$

In contrast, cloud data centers are generally assumed to have no significant constraints on GPU memory or computing capacity for executing LLMs [9]. Equation (1) constrains GPU memory for model weights only. The dynamic memory consumed during inference, including intermediate activations and the KV cache, is captured in our compute-side constraint through Eq. (3). Specifically, the per-token compute term e_m multiplied by the served tokens $\delta_{n,i,m}^t$ reflects the runtime

footprint that scales with context length and concurrency in SC-CoT, while the context-window parameter w_m and the token accumulation $K_{n,i,m}^t$ bound the effective stored context. This coupling allows dynamic serving memory to be enforced through the GPU compute and energy budget rather than an additional memory inequality.

B. CoT Inference Model

The LLMs utilize vast datasets and complex neural network architectures (like transformers) to model the joint probability distribution of sequences of text. However, rather than modeling the complete joint distribution directly (which is computationally infeasible due to its complexity and dimensionality), they approximate the marginal distribution. This is the distribution of individual tokens or phrases given their preceding context in a sequence.

Definition 1 (LLMs as marginal approximations [31]): LLMs can be considered as a universal density approximator specifically for the domain of natural language. Let $d_{i,0}$ be the task description prompt and $D_i = \{d_{i,0}, d_{i,1}, \dots, d_{i,k}\}$ be the sequence of messages in LLM agent i . The learned distribution of LLM m can be represented as p_m and the true distribution of the sequences of messages is represented as $\hat{q}$. For all sequences of messages $D_i = \{d_{i,0}, d_{i,1}, \dots, d_{i,k}\}$ of at most w_m tokens, which is limited by the context window, the learned distribution $p_m(D_i)$ follows the general product rule of probability $p_m(D_i) = p_m(d_{i,0}, d_{i,1}, \dots, d_{i,k}) = p_m(d_{i,0})p_m(d_{i,1}|d_{i,0}) \cdots p_m(d_{i,k}|d_{i,0}, d_{i,1}, \dots, d_{i,k-1})$, which is a good approximation of the true distribution $\hat{q}(D_i)$, i.e., $p_m(D_i) \approx \hat{q}(D_i)$.

Finally, LLM m generates an answer by recursively predicting the sequence of the next tokens from the learned distribution p_m , conditioned on the concatenation of the task description prompt d and the tokens sampled so far. The approximation leverages the product rule of probability to decompose the joint probability of the sequence D_i into the

product of conditional probabilities of each example $d_{i,j}$ given the previous examples $\{d_{i,0}, d_{i,1}, \dots, d_{i,j-1}\}$.

In CoT reasoning, the CoT example E_i generated from an LLM is generated based on a true context c^* and a true intention θ^* [32]. These elements represent the underlying conditions and objectives that guide the generation of the text or response by LLMs. The ambiguity of a chain of thought example is quantified by how likely it is that the true context and intention can be accurately inferred from the example. This likelihood is termed as the complement of the ambiguity.

Definition 2 (ϵ -Ambiguity [33]): The ambiguity of the chain, given the context and intention, is defined as $\hat{q}(c^*, \theta^* | E_i) = 1 - \epsilon(E_i)$. This equation states that the likelihood $\hat{q}$ of correctly identifying the context c^* and intention θ^* from the example E_i is $1 - \epsilon(E_i)$, where $\epsilon(E_i)$ represents the measure of ambiguity. The smaller the value of $\epsilon(E_i)$, the less ambiguous the example is, meaning the context and intention are more clearly communicated or inferred from E_i .

Specifically, a lower $\epsilon(E_i)$ value indicates higher clarity and lower ambiguity, making it easier to understand the intended message or instruction embedded in E_i . Conversely, a higher $\epsilon(E_i)$ indicates that the example is more ambiguous, and the true context and intention are harder to discern. To enhance the relevance and coherence of LLM agents through SC-CoT prompting, LLMs utilize a multi-path reasoning approach, where the final response is determined by reaching a consensus among various reasoning paths. This SC-CoT method samples multiple reasoning trajectories and chooses the most consistent outcome, thereby improving the robustness of the inference process. When LLMs perform inference, provided with a task description prompt d_0 , LLM m can generate $J_{i,m}$ reasoning paths for agent i by recursively predicting subsequent tokens from the learned distribution p_m , based on the concatenation of d and the previously sampled tokens.

Definition 3 (SC-CoT Reasoning [14]): Each reasoning path ρ_j results in a candidate output o_j , representing the final answer derived from the reasoning path. Thus, we obtain $J_{i,m}$ candidate outputs $\{A_1, \dots, A_J\}$ required by LLM m of agent i . For every reasoning path j , LLM m receives an SC-CoT example E_i of varying lengths $c_{j,i}$. This example $E_i = \{e_{i,0}, \dots, e_{i,k}\}$ consists of multiple thoughts, where each thought $e_{i,k}$ is made up of k_i tokens, representing an individual step in the reasoning process. The marginal probability of any output o is computed by integrating all the different reasoning paths as $P(o|d) = \sum_{j=1}^{J_{i,m}} P(\rho_j, o|d)$.

By considering all the reasoning paths, the most consistent answer o^* is the one that maximizes the marginal probability $o^* = \arg \max_o \sum_{j=1}^{J_{i,m}} P(\rho_j, o|d) = \arg \max_o \sum_{j=1}^{J_{i,m}} \mathbb{I}(o_j = o)$, where $\mathbb{I}(\cdot)$ is the identity function. When each reasoning path is equally likely, this can be simplified to finding the most frequent answer among the candidate outputs.

To derive the expected difference bound of language ambiguity, we first make the following assumptions [31], [32], [33]:

- 1) It is a uniform distribution for the true contexts c^* .
- 2) The example E_i generated from θ^* with the true context c^* is bounded by the ambiguity measure, i.e., $\epsilon(E_i) = \hat{q}(c^*, \theta^* | E_i) \leq \sigma$, where $\sigma \in [0, \frac{1}{2}]$.

- 3) For example E_i derived from true intentions θ^* with the true context c^* , the related ambiguity measure $\epsilon(E_i)$ diminishes as the length of inference sequences increases, i.e., $\lim_{l \rightarrow \infty} \epsilon(E_i) = 0$.

These assumptions posit that each possible context c^* from which language examples are generated is equally likely to occur before considering any specific examples or data. In probabilistic terms, a uniform prior distribution means that no contextual scenario is inherently more probable than any other before any additional information is taken into account. Therefore, they can make the skewness parameter $\gamma_n(c^*) = \sup_{c \in C} \frac{\hat{q}(c^*)}{\hat{q}(c_n)}$ is equal to 1 for context c_n belonged to ES n , which helps in estimating the difference in ambiguity measurements between learned distribution p_m and true distribution $\hat{q}$.

C. Performance Convergence Analysis for CoT Prompting

SC-CoT prompting can improve the capability of LLMs to handle complex reasoning, by merging multiple reasoning paths with a consensus mechanism, which ensures more reliable responses. In mobile edge networks, where latency and resources are restricted, SC-CoT is crucial to let LLMs explore various thought paths, enhancing their ability to refine responses based on broader context, thus boosting accuracy and decision-making. This approach mimics human-like problem-solving processes by breaking down complex tasks into simpler, sequential steps. CoT reasoning is particularly effective in enhancing the model's ability to tackle complex reasoning, arithmetic, commonsense, and symbolic reasoning tasks by providing a transparent and step-wise articulation of thought processes.

Lemma 1: Given a set of examples c_i with different lengths, created from the true intention θ^* and the true context c^* drawn from $q_m(c)$, let $d_{i,0}$ be the initial message or task, which is derived from $q(\cdot | \theta_0^*)$ and generated from θ^0 sampled from $q_m(\cdot | c^*)$. For any sequence of messages D_i , the following holds:

$$|p_m(D_i | d_{i,0}, E_i) - \hat{q}(D_i | d_{i,0}, c^*)| \leq \eta \prod_{y=1}^{c_i} \frac{\epsilon(E_i, y)}{1 - \epsilon(E_i, y)}, \quad (4)$$

where $\eta = 2 \frac{\epsilon(d_{i,0})}{1 - \epsilon(d_{i,0})}$ is related to the ambiguity of the input.

The proof of Lemma 1 can be found in [34] and this lemma essentially supports the concept of CoT by establishing a mathematical relationship that quantifies how the ambiguity in input influences the accuracy of the CoT reasoning process. The product term $\prod_{y=1}^{c_i} \frac{\epsilon(E_i, y)}{1 - \epsilon(E_i, y)}$ reflects the cumulative effect of errors in each step of reasoning on the overall reasoning process, emphasizing the importance of minimizing error at each reasoning step to ensure the overall reliability of the CoT reasoning process. This supports the practical implementation of CoT reasoning by highlighting the need for precise and less ambiguous reasoning steps in complex problem-solving tasks.

Furthermore, SC-CoT [14] reasoning enhances the typical chain-of-thought reasoning process by generating multiple reasoning paths and selecting the most consistent answer from those paths. This approach is based on the idea that when multiple reasoning paths converge on the same answer, it increases confidence in the correctness of that answer. This method is particularly useful for complex reasoning tasks,

where a single reasoning path might be prone to errors or bias. In SC-CoT, each reasoning path j within a CoT example i contributes independently to the formulation of the output. This formulation allows for the incorporation of diversity in reasoning strategies and the aggregation of these paths to determine the most consistent and probable outcomes.

To effectively represent the features of multiple reasoning paths in SC-CoT reasoning within a theoretical framework, we extend lemma 1 to explicitly incorporate these features and their impact on the overall reasoning process as follows.

Theorem 1: Given a set of $J_{i,m}$ SC-CoT reasoning paths sampled as $\{R_1, \dots, R_j, \dots, R_{J_{i,m}}\}$ with varying lengths, the final output is sampled from making consensus to obtain the most frequent answer under the true intention θ^* with the true context c^* drawn from $q_m(c)$, let $d_{i,0}$ be the initial input message or task, which is sampled from $q(\cdot|\theta_0^*)$ and generated from θ_0^* drawn from $q_m(\cdot|c^*)$. For any sequence of messages D_j , and with $J_{i,m}$ reasoning paths sampled as $\{R_1, \dots, R_j, \dots, R_{J_{i,m}}\}$, the following holds:

$$|p_m(D_{i,j}|d_{i,0}, E_{i,j}) - \hat{q}(D_{i,j}|d_{i,0}, c^*)| \leq \eta \prod_{j=1}^{J_{i,m}} \frac{\epsilon(E_{i,j})}{1 - \epsilon(E_{i,j})}, \quad (5)$$

where $\eta = 2\zeta_i \frac{\epsilon(d_{i,0})}{1 - \epsilon(d_{i,0})}$, which is dependent on the ambiguity of the input.

Proof: Starting from $p_m(D_{i,j}|d_{i,0}, E_{i,j})$, we sample j reasoning paths and marginalize over these paths

$$\begin{aligned} p_m(D_{i,j}|d_{i,0}, E_{i,j}) &= \frac{\sum_{j=1}^{J_{i,m}} \hat{q}(D_{i,j}, E_{i,j}, R_j)}{\sum_{j=1}^{J_{i,m}} \hat{q}(d_{i,0}, E_{i,j}, R_j)}, \end{aligned} \quad (6)$$

$$= \frac{\sum_{j=1}^{J_{i,m}} (\hat{q}(D_{i,j} \setminus \{d_{i,0}\}, E_{i,j}, R_j, c^*) + \Lambda_j)}{1 + \sum_{j=1}^{J_{i,m}} \Upsilon_j}, \quad (7)$$

where Λ_j and Υ_j are given by:

$$\Lambda_j = \sum_{c \neq c^*} \frac{\hat{q}(D_{i,j}, E_{i,j}, R_j, c)}{\hat{q}(d_{i,0}, E_{i,j}, R_j, c^*)} \quad (8)$$

and

$$\Upsilon_j = \sum_{c \neq c^*} \frac{\hat{q}(d_{i,0}, E_{i,j}, R_j, c)}{\hat{q}(d_{i,0}, E_{i,j}, R_j, c^*)}. \quad (9)$$

Using the ambiguity measure for CoT examples, we establish the bounds on Λ_j and Υ_j as

$$\Lambda_j, \Upsilon_j \leq \gamma_n^{c_{j,i}}(c^*) \frac{\epsilon(d_{i,0})}{1 - \epsilon(d_{i,0})} \prod_{y=1}^{c_{j,i}} \frac{\epsilon(E_{i,y})}{1 - \epsilon(E_{i,y})} \quad (10)$$

Finally, by combining Eqs. (14)–(18), we can get

$$\begin{aligned} &|p_m(D_{i,j}|d_{i,0}, E_{i,j}) - \hat{q}(D_{i,j}|d_{i,0}, c^*)| \\ &= \left| \frac{\sum_{j=1}^{J_{i,m}} (\hat{q}(D_{i,j} \setminus \{d_{i,0}\}, E_{i,j}, R_j, c^*) + \Lambda_j)}{1 + \sum_{j=1}^{J_{i,m}} \Upsilon_j} \right. \\ &\quad \left. - \hat{q}(D_{i,j}|d_{i,0}, c^*) \right| \leq \eta \prod_{y=1}^{c_{j,i}} \frac{\epsilon(E_{i,y})}{1 - \epsilon(E_{i,y})} \end{aligned} \quad (11)$$

where $\eta = 2\zeta_i \frac{\gamma_n^{J_{i,m}}(c^*)\epsilon(d_{i,0})}{1 - \epsilon(d_{i,0})}$, following the first assumption, indicating $\gamma_n(c^*) = 1$ and thus $\eta = 2\zeta_i \frac{\epsilon(d_{i,0})}{1 - \epsilon(d_{i,0})}$. ■

The theorem implies that the accuracy and robustness of the model's output are closely tied to the clarity and specificity of the context and intentions it is trained with. If the examples and initial inputs are unambiguous, the model's predictions will be closer to the true context, resulting in more accurate and reliable outputs. The outer product overall reasoning paths represent the compounded benefit of exploring multiple reasoning strategies. By aggregating across diverse paths, the method leverages redundancy and diversity to enhance the likelihood of converging on the correct answer, especially in complex reasoning scenarios where single-path strategies might fail. This theorem shows how the divergence from the true distribution is not only a function of the ambiguities of individual reasoning steps but also the initial conditions set by $d_{i,0}$. Effective reduction in initial ambiguity and careful management of step-wise ambiguities are crucial for enhancing the accuracy and reliability of SC-CoT systems.

Corollary 1: When examining SC-CoT examples $E_{i,j}$, for any chosen σ within the interval $[0, \frac{1}{2}]$, a length threshold $k_{i,\sigma}^* \in \mathbb{N}$ exists. For any length k_i that is greater than or equal to $k_{i,\sigma}^*$, the following condition holds true: $\epsilon(E_{i,j}) \leq \sigma$.

Proof: By selecting $\sigma \in [0, \frac{1}{2}]$, SC-CoT examples $E_{i,j}$ following Assumption 3 have the approximation that $\lim_{l \rightarrow \infty} \epsilon(E_{i,j}) = 0$. Then, there exists $k_{i,\sigma}^* \in \mathbb{N}$ such that for any $k_i \geq k_{i,\sigma}^*$, the inequality $\epsilon(E_{i,j}) \leq \sigma$ holds. ■

This proposition supports the SC-CoT approach by providing a theoretical foundation for the reduction of ambiguity with sufficiently long CoT examples, thereby enhancing the reliability and accuracy of the model's outputs.

In SC-CoT, the number of thoughts $K_{n,i,m}^t$ is accumulated during the inference process for cached LLM m to serve agent i at ES n . Depending on the caching decision $a_{n,i,m}^t$ and offloading decision $b_{n,i,m}$, the batch of inference requests executed as ES n can be calculated as $\delta_{n,i,m,j}^t = a_{n,i,m}^t b_{n,i,m}^t R_{n,i,m}^t J_{i,m} k_i$ for agent i and LLM m , where k_i is the size of each thought for agent i estimated via Corollary 1. The number of reasoning paths $J_{i,m}$ and the thought size k_i determine the inference steps and are configurable per service objective. This makes the decode length and step count variable and, when preset by the user, known at admission. Therefore, the equation to reflect the aggregation of multiple sampled paths is given as

$$K_{n,i,m}^t = \begin{cases} 0, & t = 0, \\ a_{n,i,m}^t (K_{n,i,m}^{t-1} + \delta_{n,i,m}^t), & \text{otherwise,} \end{cases} \quad (12)$$

where $\delta_{n,i,m}^t = \sum_{j=1}^{J_{i,m}} \delta_{n,i,m,j}^t$ is the total calculated tokens over J reasoning paths.

Similarly, age of thoughts (AoT) $\kappa_{n,i,m}^t$ reflect the multiple sampled paths [34] and their aggregated contributions as

$$\begin{aligned} \kappa_{n,i,m}^t &= \begin{cases} 0, & t = 0, \\ a_{n,i,m}^t \{\kappa_{n,i,m}^{t-1} + \zeta_i \delta_{n,i,m}^t - \Delta_{i,m}^t\}^+, & \text{otherwise,} \end{cases} \end{aligned} \quad (13)$$

where ζ_i is the consensus factor of SC-CoT reasoning for LLM agent i and $\Delta_{i,m}^t$ is the vanishing factor of thoughts.

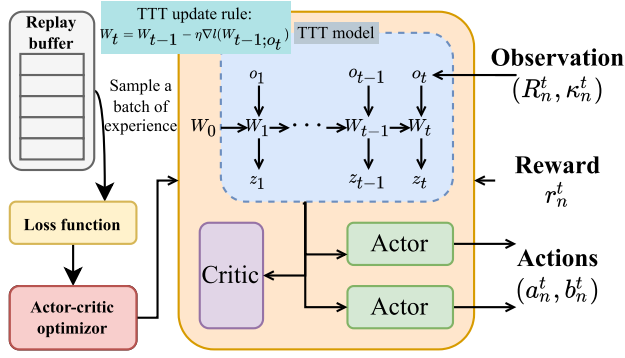


Fig. 2. The T2DRL algorithm utilizes the test-time training (TTT) model in the actor-critic network.

To reflect the SC-CoT process, we modify the performance Eq. (5) to include the most consistent answer selection mechanism as

$$A_{n,i,m}^t = \alpha_{i,m} \log \left(\frac{1}{\beta \kappa_{n,i,m}^t} \right), \quad (14)$$

where $\alpha_{i,m}$ is the zero-shot accuracy of LLM m for LLM agent i , β is the reasoning gain factor reflecting the performance improvement due to self-consistency, and $\kappa_{n,i,m}^t$ represents the adjusted AoT considering the most consistent answers selected from multiple reasoning paths.

The uniform prior over contexts and the monotone decrease of ambiguity with length are ideal assumptions used to obtain bounds and to isolate the effect of CoT sampling on distributional mismatch. Under the uniform prior, the skewness term equals one, and the constant in Eq. (11) simplifies accordingly. In practice, request distributions faced by deployed LLMs are typically non-uniform across domains and users. It helps interpret Eq. (11) by highlighting that the qualitative dependence of the bound on the initial ambiguity and the per-step ambiguity factors remains informative for non-uniform workloads as well. Likewise, the assumption that ambiguity decreases with length is intended to model the empirically observed pre-saturation regime in which additional thoughts and paths reduce error. The AoT and performance relations in Eqs. (12) to (14) capture these dynamics by accumulating contributions from multiple paths and a consensus factor, which explains the accuracy improvements we observe when adding paths or increasing the vanishing-factor setting in Fig. 5.

D. Cost Structure and Problem Formulation

1) *Cost Structure*: Edge inference costs are incurred when LLM agents are processed at ESs. These costs include switching costs from loading and evicting models in GPU memory, transmission costs for sending input prompts and results between users and ESs, computation costs for processing data at ESs, and accuracy costs due to performance gaps from using alternative LLMs. First, the switching cost arises when switching models in the GPU memory at the ESs [9]. It includes the latency and hardware wear associated with loading and evicting operations, i.e.,

$$l_{swi}^n(\mathbf{a}_n^t) = \sum_{i \in \mathcal{I}} \sum_{m \in \mathcal{M}} \lambda \cdot \mathbb{I}(a_{n,i,m}^t > a_{n,i,m}^{t-1}), \quad (15)$$

where λ is the unit cost coefficient for loading model parameters, and $\mathbb{I}(\cdot)$ is an indicator function.

The communication cost involves transmitting input prompts and inference results between users and ESs, accounting for the increased data transmission due to multiple reasoning paths, which can be calculated as

$$l_{com}^n(\mathbf{a}_n^t, \mathbf{b}_n^t) = \sum_{i \in \mathcal{I}} \sum_{m \in \mathcal{M}} R_{n,i,m}^t l_{n,i} d_i b_{n,i,m}^t, \quad (16)$$

where l_{edge}^n is the unit transmission cost for edge network access, and d_i is the input data size. Workload and cost are accounted for at the token level. For each path j , $\delta_{n,i,m,j}^t$ counts both prompt tokens and generated tokens. $R_{n,i,m}^t$ is used for arrival requests, while computation and energy scale with $\delta_{n,i,m,j}^t$.

The computation cost of executing inference requests at ES n , considering the multiple reasoning paths generated and evaluated by using SC-CoT prompting, i.e.,

$$l_{cmp}^n(\mathbf{a}_n^t, \mathbf{b}_n^t) = \sum_{i \in \mathcal{I}} \sum_{m \in \mathcal{M}} \sum_{j=1}^{J_{i,m}} \frac{\delta_{n,i,m,j}^t e_m}{f_n}, \quad (17)$$

where $\delta_{n,i,m,j}^t$ is the total computation tokens for the j -th reasoning path, e_m is the energy consumption per token, and f_n is the computing capacity of ES n . Here, e_m is model-specific, so larger models consume more per-token energy. Together with the cloud unit processing cost $l_{m,j}^0$ in Eq. (20), this captures model-size dependent costs at both edge and cloud.

The accuracy cost is incurred when there is a performance gap due to the use of alternative LLMs, considering the consensus mechanism from multiple reasoning paths as

$$l_{acc}^n(\mathbf{a}_n^t, \mathbf{b}_n^t) = \sum_{i \in \mathcal{I}} \sum_{m \in \mathcal{M}} \bar{A}_{n,i,m}^t R_{n,i,m}^t a_{n,i,m}^t b_{n,i,m}^t, \quad (18)$$

where $\bar{A}_{n,i,m}^t = \frac{1 - \alpha_{i,m}}{\kappa_{n,i,m}^t \log(1/\beta_{i,m})}$ is the accuracy cost of executing LLM m at ES n for agent i .

Overall, the total edge inference cost is the sum of these components

$$L_n^t(\mathbf{a}_n^t, \mathbf{b}_n^t) = l_{swi}^n(\mathbf{a}_n^t) + l_{com}^n(\mathbf{a}_n^t, \mathbf{b}_n^t) + l_{cmp}^n(\mathbf{a}_n^t, \mathbf{b}_n^t) + l_{acc}^n(\mathbf{a}_n^t, \mathbf{b}_n^t). \quad (19)$$

When ES n cannot handle all requests, its uncompleted inference requests are offloaded to cloud data centers, incurring cloud inference costs. This includes the costs associated with offloading requests to the cloud and processing them remotely. The cloud inference cost is calculated based on the unit processing cost at the cloud data center and the size of the data processed. The cloud inference cost includes the cost of offloading and processing these requests, accounting for the additional computation and data transmission due to multiple reasoning paths, which can be calculated as

$$L_C^t(\mathbf{a}_n^t, \mathbf{b}_n^t) = \sum_{n \in \mathcal{N}} \sum_{i \in \mathcal{I}} \sum_{m \in \mathcal{M}} \sum_{j=1}^{J_{i,m}} l_0^{m,j} (1 - b_{n,i,m}^t) R_{n,i,m}^t, \quad (20)$$

where $l_0^{m,j}$ is the unit processing cost at the cloud data center for executing reasoning path j of LLM m .

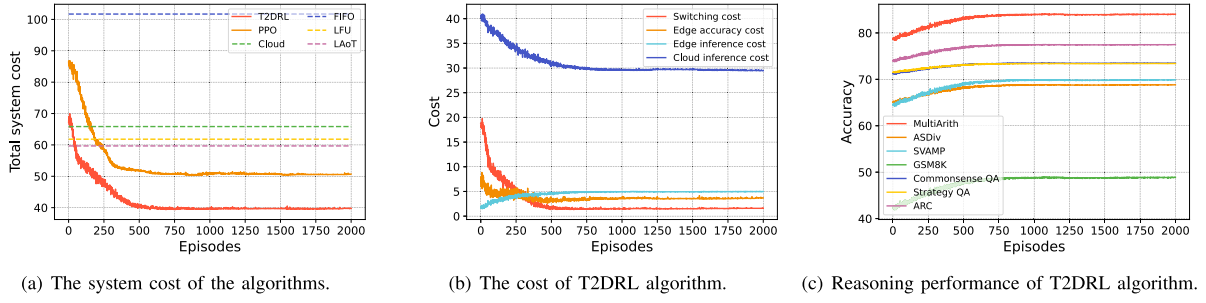


Fig. 3. Convergence analysis of the proposed T2DRL algorithm.

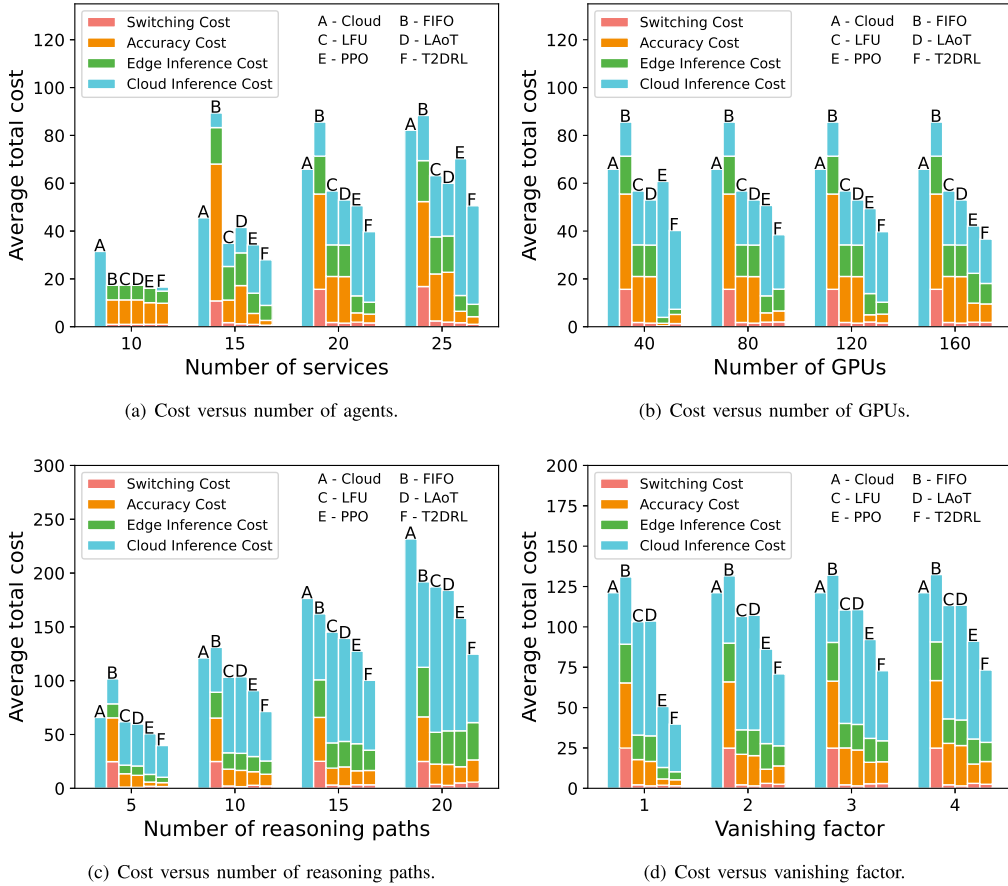


Fig. 4. Performance comparison of the proposed T2DRL algorithm under different environment settings.

The total cost for provisioning LLM agents for an ES combines both edge and cloud inference costs over a specific period. By summing these costs over time, the total cost provides a comprehensive measure of the resources and quality of LLM agents involved in operating LLM agents, helping to ensure efficient resource utilization and cost minimization while maintaining high perception and reasoning performance. Therefore, the total cost for provisioning LLM agents at ES n is the sum of the edge inference cost and the cloud inference cost over a period T as

$$L_{total}^n = \frac{1}{T} \sum_{t=1}^T (L_n^t(\mathbf{a}_n^t, \mathbf{b}_n^t) + L_C^t(\mathbf{a}_n^t, \mathbf{b}_n^t)). \quad (21)$$

However, minimizing the total system cost under hardware resource constraints and context window constraints is

NP-hard as the decision variables include discrete and continuous variables while the constraints are not linear.

IV. TEST-TIME REINFORCEMENT LEARNING-BASED MODEL CACHING AND INFERENCE OFFLOADING ALGORITHM

To optimize long-context LLM serving in mobile edge networks proactively, we propose the T2DRL algorithm by leveraging TTT models in determining model caching and inference offloading decisions. Before deployment of the algorithm, the environment needs to be formulated as an MDP consisting of state, observation, action, and reward.

A. Markov Decision Process

1) *Observation Space*: During the serving of LLM agents in the SC-CoT reasoning model, the state space captures the

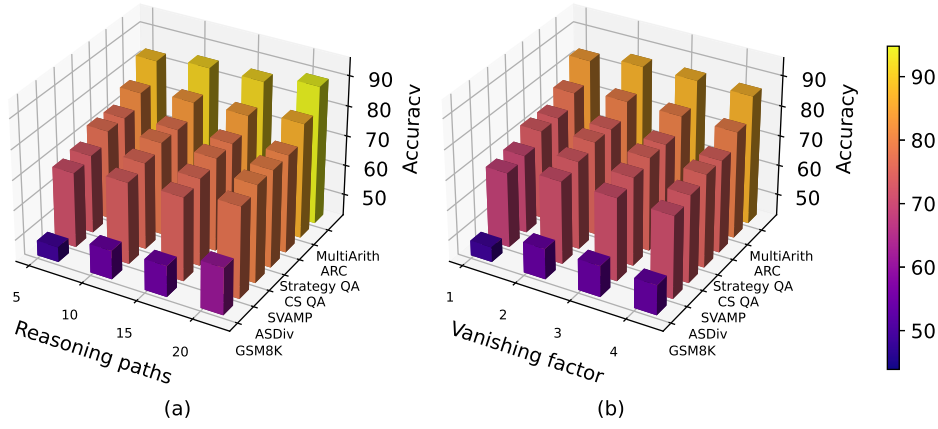


Fig. 5. Reasoning accuracy of the T2DRL algorithm under different (a) reasoning paths; (b) vanishing factor.

comprehensive status of the system, while the observation space is a subset of the state space. Therefore, the observation $O_n^t \in \mathcal{O}$ of ES n at time t is a combination of the user request information, cached model status, and model context utilization information, which can be formally defined as $O_n^t := (\mathbf{a}_n^t, R_n^t, \kappa_n^t)$, where R_n^t represents the set of all user request information; $\mathbf{a}_n^t$ represents the set of all cached model statuses, indicating whether a model is cached at a specific ES and the number of tokens utilized; and $\kappa_n^t = \{\kappa_{n,1,1}^t, \dots, \kappa_{n,I,M}^t\}$ represents the set of all model context utilization information, describing the current AoT for each cached LLM at ESs.

2) *Action Space*: The action $X_n^t \in \mathcal{X}$ in action space for the SC-CoT framework encompasses both model caching decisions $\mathbf{a}_n^t$ and inference offloading decisions $\mathbf{b}_n^t$, which can be defined as $X_n^t := (\mathbf{a}_n^t, \mathbf{b}_n^t)$.

Action feasibility layer. The actor first produces unconstrained logits for a_i^n and b_i^n . We then apply a deterministic projection to enforce feasibility with respect to the problem constraints. For GPU memory capacity, we select cache entries by descending score per unit size and stop when

$$\sum_{i,m} a_{t,n,i,m} s_m \leq G_n \quad (22)$$

as in Eq. (1). For executability, we mask offloading ratios by setting $b_{t,n,i,m} = 0$ whenever $a_{t,n,i,m} = 0$ or $R_{n,i,m}^t = 0$ to satisfy Eq. (2). For the energy budget, we reduce $1 - b_{t,n,i,m}$ in order of lowest advantage per unit energy until

$$\sum_{i,m} e_m a_{t,n,i,m} (1 - b_{t,n,i,m}) R_{n,i,m}^t \leq E_n \quad (23)$$

to satisfy Eq. (3). The environment transition uses the projected action, ensuring that all capacity and resource constraints are met at every decision step.

3) *Reward Function*: The reward function is designed to minimize the total system cost, which includes edge inference costs and cloud inference costs. The goal is to formulate a reward function that encourages actions leading to lower overall costs while maintaining high-quality LLM agents. To formulate the reward function, we aim to minimize the total system cost. The reward at time t , denoted as r_n^t , is defined as the negative of the total system cost, ensuring that lower costs yield higher rewards as $r_n^t(O_n^t, X_n^t) = -L_{total}^n(\mathbf{a}_n^t, \mathbf{b}_n^t)$, which

can effectively guide the algorithm towards minimizing the total system costs, ensuring efficient and cost-effective serving of LLM agents in the joint model caching and inference offloading framework.

B. Test-Time Deep Reinforcement Learning

As shown in Fig. 2, the T2DRL algorithm switches between exploration and exploitation by engaging with its surroundings and refining an alternative target function through random gradient ascent. The process initiates with the data collection phase, where the policy π_n is executed in the environment to gather a set of trajectories $(O_n^t, X_n^t, r_n^t, O_n^{t+1})$. Each step involves executing the current policy π to determine the action and observing the next observations and rewards. The DRL algorithm leverages a batch of trajectories comprised of sequences of states, actions, and rewards in the replay buffer, which is used to calculate advantages for policy optimization.

At the beginning of each episode, the T2DRL policy π_n consists of initialized weights W_0 for TTT models. For each input observation O_n^t in the sequence, the policy π predicts an intermediate states z_n^t as $z_n^t = \pi_n(O_n^t; W_t)$. The weights W_t are then updated using a gradient descent step on a self-supervised loss ℓ , following $W_t = W_{t-1} - \eta \nabla \ell(W_{t-1}; O_n^t)$, where the self-supervised loss can be calculated as $\ell(W; O_n^t) = \|\pi_n(\tilde{O}_n^t; W) - O_n^t\|^2$ and $\tilde{O}_n^t$ is the approximation of true state of environment. This update mechanism enables continuous learning during testing phases, allowing the model to adapt its parameters based on the immediate feedback. This process is repeated for each token in the observation sequence.

Next, advantage estimates $\tilde{A}_t$ are computed using Generalized Advantage Estimation, which effectively reduces the variance of the policy gradient estimates by leveraging the temporal difference residuals. The advantage estimates $\tilde{A}_t$ is calculated as the sum of discounted temporal differences δ_t , i.e., $\tilde{A}_t = \delta_t + (\gamma\lambda)\delta_{t+1} + \dots + (\gamma\lambda)^{T-t+1}\delta_{T-1}$, where $\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$ and the state value function $V_\theta(s_t) = \mathbb{E}_\pi[\sum_{t=0}^\infty \gamma^t r_n^t | O^0 = O]$, which estimates the expected return from state O^t under the current policy $\pi(\theta)$.

Following self-supervised loss and advantage estimation, the TTT model-based policy can output the action $X_n^t = \pi(O_n^t; \theta; W_t)$ for the observed. The model updates its hidden

state using the trajectory data and performs test-time training by updating its weights with a self-supervised loss function ℓ . This adaptation process enables the model to tune its parameters specifically to the nuances of the encountered test data, enhancing its predictive accuracy and responsiveness. Furthermore, the policy of T2DRL is then updated by defining the PPO surrogate loss with the clipped objective.

Finally, the policy parameters θ and TTT model parameters W are updated with the optimized values. This iterative process continues until convergence or for a fixed number of iterations. By utilizing TTT models in the actor-critic network, the T2DRL algorithm can adapt its learned policy and model parameters during test phases, enhancing the system's ability to efficiently manage model caching and inference offloading. This optimization leads to improved resource utilization and minimized operational costs while ensuring high-quality LLM agents, making the T2DRL framework a robust and adaptive solution for long-context LLM serving in mobile edge networks.

Let J be the number of sampled reasoning paths, k be the average tokens per path, b be the TTT minibatch size, d be the hidden width of the TTT learner, and T_{iter} be the number of price-clock iterations in the optional market layer with n buyers and m sellers. Per decision step, T2DRL incurs $O(Jk)$ token evaluations for SC-CoT with $O(J)$ aggregation; a lightweight TTT inner loop of $O(bd^2)$ for the weight update plus $O(b^2d)$ for minibatch readout in a dual form that scales linearly with sequence length [16]; the same actor-critic passes as PPO for a matched backbone and batch size; and, if market clearing is enabled, $O(T_{\text{iter}}(n + m))$. For the same observation space and network, PPO omits the SC-CoT and TTT terms. Despite these additions, T2DRL achieves a lower end-to-end system cost than PPO in our experiments because higher reasoning accuracy and online adaptation reduce cloud offloading and switching.

V. MARKET FORMULATION AND TRUTHFUL DOUBLE DUTCH AUCTION MECHANISM DESIGN

For serving long-context LLMs at mobile edge networks, the auction market is modeled as a dynamic interaction between resource-limited edge servers (sellers) and user devices (buyers). Formulating a long-context LLM serving market at mobile edge networks is essential to efficiently allocate limited edge resources, optimize costs, and handle dynamic user demands. It provides a structured framework for balancing trade-offs between computational efficiency, accuracy, and latency while ensuring fairness and transparency in resource sharing. By enabling real-time adaptation and incentivizing truthful participation, the market framework maximizes social welfare and scalability, supporting diverse applications in resource-constrained edge environments. In the long-context LLM serving market, there is an auctioneer who coordinates the market to optimize the allocation of resources and inference tasks. We price a small, time-sliced unit as either one additional reasoning path or a fixed bundle of tokens that produces one path on average. For buyer i , the marginal value of a unit equals the expected accuracy gain from adding the next path times the request utility. For seller j , the marginal

cost equals the incremental GPU time, energy, and KV-cache opportunity cost at the current load. This mapping turns per-request test-time scaling into a two-sided quantity market with concave buyer values and nondecreasing seller costs.

A. Long-Context LLM Serving Market Formulation

The long-context LLM serving market consists of three primary entities:

- **Buyers (User Devices):** Each buyer i submits a buy-bid, k_i^t , representing the maximum price they are willing to pay for LLM inference services. The valuation v_i^b depends on the required quality Q_i and resource demand R_i , given as:

$$v_i^b = Q_i l_{acc}^i, \quad (24)$$

where Q_i reflects the inference quality coefficient of buyer i .

- **Sellers (Edge Servers):** Each seller j submits a sell-bid, o_j^t , indicating the minimum price they are willing to accept. This valuation v_j^s is based on resource costs, including memory and computation, defined as:

$$v_j^s = G_j(l_{swi}^j + l_{com}^j + l_{cmp}^j), \quad (25)$$

where G_j is server j resource utilization coefficient.

- **Auctioneer:** Operates the DDA mechanism, coordinating buyer and seller clocks to identify the market-clearing price, p^* .

We interpret v_i^b and v_j^s as marginal willingness to pay and marginal willingness to accept per unit, denominated in cost units consistent with the switching, transmission, computation, and accuracy terms in Section III-D, which makes the mechanism compatible with the system cost in Eq. (21).

B. Auction Mechanism Design

The t-DDA keeps the Double Dutch interaction but replaces the final clearing step by a trade-reduction rule that yields dominant-strategy incentive compatibility. The auction process has four stages.

1) *Clock Broadcast:* Two synchronized clocks govern price discovery:

- **Buyer clock** C_B^t starts at a high value $p_{\max}$ and decreases by a step Θ^t .
- **Seller clock** C_S^t starts at a low value $p_{\min}$ and increases by a step Θ^t .

2) *Clock Acceptance:* An inactive buyer i may accept the buyer clock when it reaches a favorable price. Let the buyer stop price be

$$b_i \triangleq \text{the price } C_B^t \text{ when buyer } i \text{ presses accept.} \quad (26)$$

An inactive seller j may accept the seller clock when it reaches a favorable price. Let the seller stop price be

$$a_j \triangleq \text{the price } C_S^t \text{ when seller } j \text{ presses accept.} \quad (27)$$

Define the acceptance sets at time t as $\mathcal{M}_B^t$ and $\mathcal{N}_S^t$ as in the baseline design.

3) *Clock Adjustment*: If no new acceptances arrive, the auctioneer narrows the gap by

$$C_B^{t+1} = C_B^t - \Theta^t, \quad C_S^{t+1} = C_S^t + \Theta^t, \quad (28)$$

until termination at T when no further acceptances are expected or the clocks cross.

4) *Truthful Clearing (Trade-Reduction Rule)*: At termination, collect stop prices $\{b_i\}$ and $\{a_j\}$. Sort them with fixed tie-breaking: $b_{(1)} \geq b_{(2)} \geq \dots$, $a_{[1]} \leq a_{[2]} \leq \dots$.

Set sentinels: $b_{(0)} = +\infty$, $a_{[0]} = 0$. Define $b_{(k^*+1)} = 0$ and $a_{[k^*+1]} = +\infty$ if those terms are needed but undefined.

$$k^* \triangleq \max\{k \geq 0 : b_{(k)} \geq a_{[k]}\}. \quad (29)$$

If $k^* = 0$, set $q^* = 0$ and stop (no trade). Otherwise define

$$p^{\text{mid}} = \frac{b_{(k^*+1)} + a_{[k^*+1]}}{2}. \quad (30)$$

The *allocation* and *pricing* rules are shown in (26) and (27).

The *allocation* and the *pricing* rules of t-DDA are: (31) and (32), as shown at the bottom of the page.

Case A clears at a single uniform price p^{mid} . Case B removes the pivotal trade and charges winners prices determined only by the highest losing buyer and the lowest losing seller. This step is what guarantees dominant-strategy truthfulness.

6) *Utilities and Welfare*: Let p_i be the price paid by buyer i when she wins, and r_j the amount received by seller j when he wins. Assuming quasi-linear utilities and truthful reports:

$$u_i = \begin{cases} v_i^b - p_i, & \text{if buyer } i \text{ wins,} \\ 0, & \text{otherwise,} \end{cases} \quad (33)$$

and

$$\tilde{u}_j = \begin{cases} r_j - v_j^s, & \text{if seller } j \text{ wins,} \\ 0, & \text{otherwise.} \end{cases} \quad (34)$$

Let Π be the auctioneer surplus, equal to total buyer payments minus total seller receipts. Then

$$S W_{\text{total}} = \sum u_i + \sum \tilde{u}_j + \Pi, \quad (35)$$

where

$$\Pi = \begin{cases} 0, & \text{if Case A,} \\ (k^* - 1)(b_{(k^*)} - a_{[k^*]}), & \text{if Case B.} \end{cases} \quad (36)$$

C. Property Analysis

We prove dominant-strategy incentive compatibility, ex post individual rationality, and budget balance for the truthful Double Dutch clearing rule in Eqs. (26) and (27). Throughout, agents are single-parameter with quasi-linear utilities, values and costs are private and independent, and tie-breaking is fixed ex ante and does not depend on any single agent's report.

We prove DSIC by establishing bid-monotonicity and the existence of report-independent critical prices for winners.

Lemma 2: [Monotonicity] Fix the reports of all agents except one buyer i . Then i 's allocation indicator $x_i(b_i) \in \{0, 1\}$ is weakly increasing in her report b_i . Symmetrically, the allocation indicator $y_j(a_j)$ for seller j is weakly increasing as a_j decreases.

Proof: Raising a buyer's report can only improve her rank among buyers, hence cannot remove her from a winner set that is defined by being among the top k or top $k-1$ ranks. Lowering a seller's report can only improve his rank among sellers, hence cannot remove him from a winner set defined by being among the bottom k or bottom $k-1$ ranks. The case selection between (31) and (32) depends on losing margins $b_{(k^*+1)}$ and $a_{[k^*+1]}$ and on $b_{(k^*)}$, $a_{[k^*]}$, which do not involve any winning agent's own report. Therefore the winner indicators are monotone in the respective reports. ■

Lemma 3: [Critical prices] Fix all reports except buyer i 's. There exists a threshold τ_i that depends only on the other agents' reports, such that

$$x_i(b_i) = \mathbf{1}\{b_i \geq \tau_i\}, \quad p_i = \begin{cases} \tau_i, & \text{if } x_i(b_i) = 1, \\ 0, & \text{if } x_i(b_i) = 0, \end{cases} \quad (37)$$

and p_i is independent of b_i whenever $x_i(b_i) = 1$. An analogous statement holds for each seller j with a threshold ρ_j where $y_j(a_j) = \mathbf{1}\{a_j \leq \rho_j\}$ and $r_j = \rho_j$ when $y_j(a_j) = 1$.

Proof: Consider a profile in which buyer i is a winner.

Case A: the transaction price is p^{mid} which is the average of the top losing buyer's bid $b_{(k^*+1)}$ and the top losing seller's ask $a_{[k^*+1]}$. Since i is a winner, neither statistic equals b_i . Therefore p^{mid} is independent of b_i . Moreover, i is a winner if and only if $b_i \geq p^{\text{mid}}$ by construction of k^* and the inequality $a_{[k^*]} \leq p^{\text{mid}} \leq b_{(k^*)}$. Set $\tau_i = p^{\text{mid}}$.

Case B: winners are the top k^*-1 buyers, each charged $b_{(k^*)}$ which is the highest losing bid. Since i is a winner, $b_{(k^*)}$ is not b_i , hence is independent of b_i . A buyer wins if and only if her bid is at least $b_{(k^*)}$. Set $\tau_i = b_{(k^*)}$.

The seller side is symmetric. In Case A each winning seller is paid p^{mid} which is independent of any winner's report and a seller wins if and only if $a_j \leq p^{\text{mid}}$, so $\rho_j = p^{\text{mid}}$. In Case B

Case A: $a_{[k^*]} \leq p^{\text{mid}} \leq b_{(k^*)}$.

$q^* = k^*$ units traded between top k^* buyers and bottom k^* sellers.

Each winner transacts at price $P_B^* = P_S^* = p^{\text{mid}}$.

(31)

Case B: (otherwise).

$q^* = k^* - 1$ units traded between top $k^* - 1$ buyers and bottom $k^* - 1$ sellers.

Buyers pay $P_B^* = b_{(k^*)}$, sellers receive $P_S^* = a_{[k^*]}$.

(32)

each winning seller is paid $a_{[k^*]}$ which is the lowest losing ask and is independent of any winner's report, and a seller wins if and only if $a_j \leq a_{[k^*]}$, so $\rho_j = a_{[k^*]}$. ■

Theorem 2: [Dominant-strategy incentive compatibility] Truthful reporting is a dominant strategy for every buyer and every seller under the clearing rule in (31)–(32).

Proof: By Lemma 2 the allocation is monotone in each agent's report. By Lemma 3 each winning agent pays or receives a report-independent critical price that equals the threshold at which the allocation switches for that agent. For a buyer with true value v_i the utility is

$$u_i(b_i; v_i) = \begin{cases} v_i - \tau_i, & \text{if } b_i \geq \tau_i, \\ 0, & \text{if } b_i < \tau_i, \end{cases} \quad (38)$$

which is maximized by reporting any $b_i \geq v_i$ if $v_i \geq \tau_i$ and any $b_i < v_i$ if $v_i < \tau_i$. Hence truthful reporting is a dominant strategy. The proof for sellers is identical with costs and thresholds (c_j, ρ_j) . ■

Theorem 3: [Individual rationality] Every winning buyer i obtains $u_i \geq 0$ and every winning seller j obtains $\tilde{u}_j \geq 0$.

Proof: Case A: $P^* = p^{\text{mid}} \in [a_{[k^*]}, b_{(k^*)}]$. A winning buyer is among the top k^* buyers, so $b_i \geq b_{(k^*)} \geq P^*$. Under dominant strategies, a buyer accepts only at prices no greater than her value, so $v_i \geq b_i \geq P^*$ and $u_i = v_i - P^* \geq 0$. A winning seller is among the bottom k^* sellers, so $a_j \leq a_{[k^*]} \leq P^*$. Under dominant strategies a seller accepts only at prices no lower than his cost, so $P^* - c_j \geq 0$ and $\tilde{u}_j \geq 0$.

Case B: winners are strictly infra-marginal relative to the losing pair that defines $b_{(k^*)}$ and $a_{[k^*]}$. Each winning buyer pays $P_B^* = b_{(k^*)}$ and satisfies $b_i \geq b_{(k^*-1)} \geq b_{(k^*)}$ hence $v_i \geq b_i \geq P_B^*$, so $u_i \geq 0$. Each winning seller receives $P_S^* = a_{[k^*]}$ and satisfies $a_j \leq a_{[k^*-1]} \leq a_{[k^*]} \leq P_S^*$, so $\tilde{u}_j \geq 0$. ■

Theorem 4: [Budget balance] The mechanism is strongly budget balanced in Case A and weakly budget balanced in Case B. Therefore the auctioneer never runs a deficit.

Proof: Case A: $q^* = k^*$ trades clear at the common price $P^* = p^{\text{mid}}$. Total buyer payments equal $k^* p^{\text{mid}}$ and total seller receipts equal $k^* p^{\text{mid}}$, so the auctioneer's net transfer is zero.

Case B: $q^* = k^* - 1$ trades execute. Total buyer payments equal $(k^* - 1) b_{(k^*)}$ and total seller receipts equal $(k^* - 1) a_{[k^*]}$. The auctioneer's net surplus is

$$(k^* - 1) (b_{(k^*)} - a_{[k^*]}) \geq 0, \quad (39)$$

so no deficit occurs. Hence, the mechanism is weakly budget-balanced. ■

If a strong budget balance is required in all profiles, one may rebate the Case B surplus using a rule that is independent of each agent's own report. Such rebates preserve dominant-strategy incentive compatibility but must be designed to avoid deficits in finite samples. This refinement is optional and does not affect Theorems 2–4.

1) *Efficiency:* The traded quantity equals the efficient k^* in Case A and $k^* - 1$ in Case B, so the mechanism sacrifices at most one marginal trade to obtain truthfulness. This loss vanishes in large markets under standard regularity.

2) *Complexity:* The clock phase has the same complexity as in the baseline DDA. The clearing step adds a sort of stop prices, which is $O((n + m) \log(n + m))$, and constant-time

computation of k^* and p^{mid} . The overall online complexity is $O(T \cdot (n + m)) + O((n + m) \log(n + m))$, where T is the number of clock ticks.

VI. EXPERIMENTAL RESULTS

In the experimental setup, we consider a mobile edge system configured with varying parameters to evaluate the proposed framework and algorithm. The ESs are equipped with multiple GPUs with a total memory of 80 GB and a computing capability of 312,000 GFLOPs, supported by an energy cap of 300 W per GPU. For constructing LLM agents with perception and reasoning capability, we use the Imagebind [35] as the perception module. In addition, LaMDA-137B and PaLM-540B [14] are adopted in the reasoning module to handle different requests. Furthermore, the network access costs are set to 0.0001 for edge access and 0.0075 for cloud access. The switching cost coefficient is set to 10^{-5} and the accuracy cost coefficient is set to 2.5. We vary input context length between 100 and 200 tokens and study different $J_{i,m}$ to reflect token-level and step-level variability. The system is simulated for 100 steps to thoroughly test the performance of the proposed algorithm under dynamic network conditions. The detailed reasoning performance of LaMDA-137B and PaLM-540B is listed in Table I and Table II.

In the configuration of the PPO and T2DRL, the learning rate is set to 3×10^{-4} and the discount factor is set to 0.95. We conducted the experiments with the seed of 42 to ensure reproducibility. We trained PPO and T2DRL over 2000 epochs, with each epoch consisting of 2000 steps. The actor-critic network of PPO utilizes hidden layers set at 128 units. The batch size is set at 128. Specific PPO adjustments included a value function coefficient of 0.25, an entropy coefficient of 0, and clipping epsilon at 0.2 for the policy updates. Additionally, the gradient is norm-capped at 0.5, with a GAE-lambda of 0.95 for advantage estimation. The TTT model is configured with a hidden size of 32, an intermediate size of 32, one hidden layer, four attention heads, and a mini-batch size of four.

We compare our method against five baselines implemented under the same traces, hardware budgets, and cost definitions.

- **Cloud-only.** All requests are offloaded to the cloud. No model is cached on edge GPUs, and no switching cost is incurred.
- **FIFO cache.** On a cache miss, the earliest loaded model is evicted first until enough memory is freed. This is the classic first-in first-out policy.
- **LFU cache.** On a cache miss, the least frequently requested model is evicted first, with ties broken by recency. This is the standard least-frequently used policy.
- **LAoT cache [34].** We use the Age of Thoughts defined in the paper to prefer models whose recent thoughts are freshest. When space is needed, evict the model with the largest age so that recently used thought caches remain resident.
- **PPO scheduler [15].** A learning-based scheduler that optimizes caching and offloading actions with proximal policy optimization (PPO) under the same network and cost settings as our method, but without test-time training adaptation.

The proposed T2DRL algorithm demonstrates robust performance in reducing system costs and enhancing reasoning accuracy over 2000 episodes, as illustrated in Fig. 3. In Fig. 3(a), the T2DRL algorithm shows a rapid decline in total system costs during the initial episodes, achieving convergence at approximately 500 episodes. Compared to baseline algorithms such as FIFO, LFU, and Cloud, T2DRL achieves at least a 30% reduction in system costs, while outperforming the PPO algorithm by 20% in cost efficiency after convergence. This significant reduction highlights the algorithm's effectiveness in dynamic resource allocation and context-aware inference optimization. Fig. 3(b) provides a breakdown of cost components, including switching costs, edge accuracy costs, edge inference costs, and cloud inference costs. The results reveal that T2DRL effectively minimizes switching costs and balances the trade-offs between edge and cloud inference costs, stabilizing these values as the episodes progress. Notably, cloud inference costs experience a sharp decline early on, indicating the algorithm's ability to prioritize edge-based solutions for improved cost efficiency. The edge accuracy cost remains stable after convergence, reflecting the algorithm's ability to maintain high inference quality while managing resources efficiently. Furthermore, Fig. 3(c) highlights the reasoning performance of the T2DRL algorithm across various datasets, such as MultiArith, ASDiv, SVAMP, GSM8K, and Commonsense QA. The reasoning accuracy consistently improves throughout the training process, reaching a peak by the final episodes. T2DRL achieves the highest accuracy across all tested tasks, demonstrating its capability to handle complex reasoning tasks effectively. This improvement underscores the algorithm's ability to adaptively leverage context windows and optimize long-context LLM serving. In conclusion, the convergence analysis, as detailed in Fig. 3, underscores the T2DRL algorithm's ability to significantly reduce system costs while consistently enhancing the reasoning performance of LLM agents, making it a highly effective framework for dynamic and resource-constrained mobile edge environments.

We now report raw task accuracy for Cloud, FIFO, LFU, LAoT, PPO, and T2DRL under identical SC-CoT reasoning settings, holding LLMs, prompts, reasoning paths, and vanishing-factor grids fixed, so that reasoning performance across algorithms can be compared directly. Figure 4 showcases the superior performance of the T2DRL algorithm compared to various baseline methods, including Cloud, FIFO, LFU, Least Age-of-Thought (LAoT), and PPO, across diverse system configurations. As shown in Fig. 4(a), T2DRL consistently achieves the lowest total system cost as the number of agents increases from 10 to 25. This significant cost reduction is attributed to T2DRL's effective management of multiple agents, which minimizes switching costs while optimizing edge and cloud inference costs. In contrast, baseline algorithms such as FIFO and Cloud exhibit substantial increases in costs as the number of agents grows, highlighting T2DRL's scalability and resource efficiency in high-demand scenarios. Fig. 4(b) demonstrates that T2DRL effectively reduces total costs as the number of GPUs increases from 40 to 160. This result underscores T2DRL's ability to adapt to additional computational resources by efficiently distributing

workloads and leveraging GPU capacity. Compared to baseline methods, T2DRL achieves consistently lower costs across all GPU configurations, with significant savings observed in edge inference costs and reduced reliance on cloud inference. When analyzing the number of reasoning paths, as depicted in Fig. 4(c), T2DRL shows remarkable cost savings as the number of paths increases from 5 to 20. The algorithm optimally manages reasoning paths by reducing switching and accuracy costs, which dominate in scenarios requiring complex multi-path reasoning. While the baseline algorithms struggle to maintain efficiency with an increasing number of paths, T2DRL demonstrates resilience and maintains low total system costs. Lastly, Fig. 4(d) evaluates the performance of T2DRL under varying vanishing factors, which represent the diminishing relevance of contextual information over time. T2DRL exhibits the lowest average total cost across all vanishing factor settings, effectively mitigating the impact of context loss by balancing edge inference and switching costs. In comparison, baseline algorithms like Cloud and FIFO display higher costs due to inefficient context management and over-reliance on cloud resources. Overall, the experimental results in Fig. 4 highlight T2DRL's robust adaptability and efficiency across different environmental settings. By minimizing switching, accuracy, edge inference, and cloud inference costs, T2DRL significantly outperforms existing baselines in scenarios with varying agent numbers, GPU availability, reasoning paths, and vanishing factors. These findings underscore T2DRL's potential as a highly effective solution for dynamic and resource-constrained mobile edge networks.

Specifically, Fig. 5 illustrates the reasoning performance of the T2DRL algorithm tested on real-world datasets, including MultiArith [36], ARC [37], Strategy QA [38], Commonsense QA [39], SVAMP [40], ASDiv [41], and GSM8K [42], showing that as the number of reasoning paths increases from 5 to 20 in Fig. 5(a), and the vanishing factor rises from 1 to 4 in Fig. 5(b), the accuracy consistently improves across various datasets. As illustrated in Fig. 5(a), the reasoning accuracy consistently improves as the number of reasoning paths increases from 5 to 20 across all datasets. The datasets with simpler reasoning requirements, such as GSM8K and SVAMP, show moderate improvements, while datasets like MultiArith and ARC, which demand more complex reasoning, exhibit steeper accuracy gains with additional paths. This trend highlights T2DRL's ability to leverage multi-path reasoning effectively, balancing computational overhead with improved inference quality. The results indicate that T2DRL dynamically optimizes resource allocation for reasoning paths, enabling the LLM agents to explore diverse reasoning chains and enhance overall performance. In Fig. 5(b), reasoning accuracy improves with an increase in the vanishing factor, which reflects the relevance of context over time. Datasets that rely heavily on maintaining contextual consistency, such as Strategy QA and Commonsense QA, see the most significant improvements. As the vanishing factor rises from 1 to 4, T2DRL demonstrates a remarkable ability to maintain and utilize context effectively, preventing degradation in inference quality. This result underscores the algorithm's robustness in scenarios where context relevance decays over time, as it adapts to mitigate information

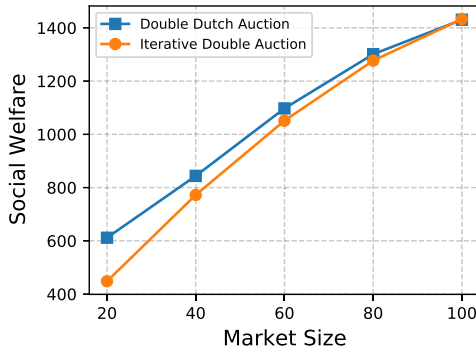


Fig. 6. Social welfare verse market size.

loss and prioritize critical reasoning components. Across both subfigures, T2DRL achieves high accuracy consistently, with MultiArith and ARC emerging as the top-performing datasets, achieving accuracy levels close to 90% in favorable conditions. This pattern indicates that T2DRL is well-suited for handling a wide variety of reasoning tasks, from numerical and logical reasoning (MultiArith) to commonsense and contextual reasoning (Commonsense QA, Strategy QA). These trends are consistent with the previous theoretical view that, within the operating regime considered by our assumptions, longer examples and more reasoning paths reduce ambiguity and improve accuracy.

To evaluate the long-context LLM serving market and proposed auction mechanism, Fig. 6 demonstrates the social welfare performance of the Iterative Double Auction (IDA) and DDA across varying market sizes. Both mechanisms exhibit a consistent upward trend in social welfare as market size increases, reflecting their ability to allocate resources more efficiently with greater market activity. However, the DDA consistently outperforms the Iterative Double Auction across all market sizes, showcasing its superior efficiency and scalability. At smaller market sizes (e.g., 20 and 40), the DDA achieves the social welfare approximately 36% higher than IDA, indicating its ability to optimize outcomes even in resource-constrained or limited-participant scenarios. As the market size grows to 60, 80, and eventually 100, the gap between the two mechanisms narrows slightly. Despite this convergence, the DDA maintains a clear advantage, achieving higher social welfare values across the board. For instance, at a market size of 100, the DDA outperforms the IDA by approximately 12%, demonstrating its continued robustness and adaptability in larger, more complex markets. The superior performance of the DDA can be attributed to its dynamic and synchronized pricing mechanism, which effectively matches supply and demand while maximizing participant utility. This ensures optimal resource allocation and minimizes inefficiencies, particularly in scenarios with heterogeneous market participants and varying resource demands.

VII. CONCLUSION

In this paper, we have proposed a universal long-context LLM serving framework for supporting the perception and complex reasoning of LLM agents in mobile edge net-

works, which leverages the T2DRL algorithm to optimize model caching and inference offloading adaptively. Specifically, we have designed the system model to optimize the deployment and execution of long-context LLMs in mobile edge networks by dynamically managing model caching and inference offloading, considering resource constraints such as GPU memory and computational power. The theoretical analysis introduces bounds on ambiguity and error, assessing the system's capability to maintain accuracy and reliability under varied conditions, particularly in handling long-context demands efficiently. The proposed T2DRL algorithm can learn and adapt the optimal policy in real-time, optimizing caching and inference offloading proactively to reduce system costs and ensure the perception and reasoning performance of LLM agents in dynamic edge systems. Additionally, we have leveraged the DDA mechanism for resource allocation by dynamically matching supply and demand, ensuring fairness and transparency while maximizing social welfare. Experimental results have highlighted the effectiveness of the proposed algorithm in reducing total system cost for LLM serving in mobile edge networks. In future work, we plan to enhance topology generalization by augmenting the observation with permutation-invariant neighborhood summaries or a lightweight graph encoder, trained with topology randomization, while retaining the same feasibility projection. For model generalization, we will parameterize actions by model descriptors rather than model IDs, so that unseen LLMs can be incorporated at inference by supplying their descriptors. We will derive more general and quantitative convergence guarantees for the ambiguity-reduction analysis by relaxing the uniform-prior assumption and by extending the DDA discussion to asynchronous/event-driven clearing. We will explicitly model peak inference-time GPU memory, e.g., KV cache and activation buffers, as an additional capacity constraint, since this peak footprint may deviate from the compute/energy proxy.

REFERENCES

- [1] M. Xu et al., "When large language model agents meet 6G networks: Perception, grounding, and alignment," *IEEE Wireless Commun.*, vol. 31, no. 6, pp. 63–71, Dec. 2024.
- [2] T. B. Brown et al., "Language models are few-shot learners," in *Proc. NIPS*, 2020, pp. 1877–1901.
- [3] S. Bubeck et al., "Sparks of artificial general intelligence: Early experiments with GPT-4," 2023, *arXiv:2303.12712*.
- [4] H. Zhou et al., "Large language model (LLM) for telecommunications: A comprehensive survey on principles, key techniques, and opportunities," *IEEE Commun. Surveys Tuts.*, vol. 27, no. 3, pp. 1955–2005, Jun. 2025.
- [5] P. Zeng et al., "The CAP principle for LLM serving," 2024, *arXiv:2405.11299*.
- [6] S. Ye et al., "Galaxy: A resource-efficient collaborative edge AI system for in-situ transformer inference," in *Proc. IEEE INFOCOM Conf. Comput. Commun.*, May 2024, pp. 1001–1010.
- [7] L. Pan, L. Wang, S. Chen, and F. Liu, "Retention-aware container caching for serverless edge computing," in *Proc. IEEE INFOCOM Conf. Comput. Commun.*, May 2022, pp. 1069–1078.
- [8] Y. Liu, L. Su, C. Joe-Wong, S. Ioannidis, E. Yeh, and M. Siew, "Cache-enabled federated learning systems," in *Proc. 24th Int. Symp. Theory, Algorithmic Found., Protocol Design Mobile Netw. Mobile Comput.*, Oct. 2023, pp. 1–11.
- [9] K. Zhao et al., "EdgeAdaptor: Online configuration adaption, model selection and resource provisioning for edge DNN inference serving at scale," *IEEE Trans. Mobile Comput.*, vol. 22, no. 10, pp. 5870–5886, Oct. 2023.

- [10] X. Ma, A. Zhou, S. Zhang, and S. Wang, "Cooperative service caching and workload scheduling in mobile edge computing," in *Proc. IEEE INFOCOM Conf. Comput. Commun.*, Jul. 2020, pp. 2076–2085.
- [11] C. Hu et al., "MemServe: Context caching for disaggregated LLM serving with elastic memory pool," 2024, *arXiv:2406.17565*.
- [12] S. Singh et al., "LLM-dCache: Improving tool-augmented LLMs with GPT-driven localized data caching," in *Proc. 31st IEEE Int. Conf. Electron., Circuits Syst. (ICECS)*, Nov. 2024, pp. 1–4.
- [13] J. Wei et al., "Chain-of-thought prompting elicits reasoning in large language models," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 35, 2022, pp. 24824–24837.
- [14] X. Wang et al., "Self-consistency improves chain of thought reasoning in language models," 2022, *arXiv:2203.11171*.
- [15] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," 2017, *arXiv:1707.06347*.
- [16] Y. Sun et al., "Learning to (learn at test time): RNNs with expressive hidden states," 2024, *arXiv:2407.04620*.
- [17] Z. Lin, G. Qu, Q. Chen, X. Chen, Z. Chen, and K. Huang, "Pushing large language models to the 6G edge: Vision, challenges, and opportunities," *IEEE Commun. Mag.*, vol. 63, no. 9, pp. 52–59, Sep. 2025.
- [18] Z. Yang, Y. Yang, C. Zhao, Q. Guo, W. He, and W. Ji, "PerLLM: Personalized inference scheduling with edge-cloud collaboration for diverse LLM services," 2024, *arXiv:2405.14636*.
- [19] N. Su, C. Hu, B. Li, and B. Li, "Titanic: Towards production federated learning with large language models," in *Proc. IEEE INFOCOM Conf. Comput. Commun.*, May 2024, pp. 611–620.
- [20] X. Yuan, H. Li, K. Ota, and M. Dong, "Generative inference of large language models in edge computing: An energy efficient approach," in *Proc. Int. Wireless Commun. Mobile Comput. (IWCMC)*, May 2024, pp. 244–249.
- [21] R. Zhou, X. Wu, H. Tan, and R. Zhang, "Two time-scale joint service caching and task offloading for UAV-assisted mobile edge computing," in *Proc. IEEE INFOCOM Conf. Comput. Commun.*, May 2022, pp. 1189–1198.
- [22] G. Qu et al., "TrimCaching: Parameter-sharing edge caching for AI model downloading," 2024, *arXiv:2404.14204*.
- [23] Z. Wang, M. Li, L. Zhao, H. Zhou, and N. Wang, "A3C-based computation offloading and service caching in cloud-edge computing networks," in *Proc. IEEE INFOCOM Conf. Comput. Commun. Workshops (INFOCOM WKSHPS)*, May 2022, pp. 1–2.
- [24] A. Narayanan, S. Verma, E. Ramadan, P. Babaie, and Z.-L. Zhang, "DeepCache: A deep learning based framework for content caching," in *Proc. Workshop Netw. Meets AI ML-NetAI*, 2018, pp. 48–53.
- [25] M. Xie, J. Ye, G. Zhang, and X. Ni, "Deep reinforcement learning-based computation offloading and distributed edge service caching for mobile edge computing," *Comput. Netw.*, vol. 250, Aug. 2024, Art. no. 110564.
- [26] Y. Zhao, R. Li, C. Wang, X. Wang, and V. C. M. Leung, "Neighboring-aware caching in heterogeneous edge networks by actor-attention-critic learning," in *Proc. IEEE Int. Conf. Commun.*, Jun. 2021, pp. 1–6.
- [27] Z. Yao, S. Xia, Y. Li, and G. Wu, "Cooperative task offloading and service caching for digital twin edge networks: A graph attention multi-agent reinforcement learning approach," *IEEE J. Sel. Areas Commun.*, vol. 41, no. 11, pp. 3401–3413, Nov. 2023.
- [28] Y. He, J. Fang, F. R. Yu, and V. C. Leung, "Large language models (LLMs) inference offloading and resource allocation in cloud-edge computing: An active inference approach," *IEEE Trans. Mobile Comput.*, vol. 23, no. 12, pp. 11253–11264, Dec. 2024.
- [29] K. A. McCabe, S. J. Ramenti, and V. L. Smith, "Designing call auction institutions: Is double Dutch the best?," *Econ. J.*, vol. 102, no. 410, pp. 9–23, Jan. 1992.
- [30] M. Xu, D. Niyato, J. Kang, Z. Xiong, C. Miao, and D. I. Kim, "Wireless edge-empowered metaverse: A learning-based incentive mechanism for virtual reality," in *Proc. IEEE Int. Conf. Commun.*, May 2022, pp. 5220–5225.
- [31] C. Yun, S. Bhojanapalli, A. S. Rawat, S. J. Reddi, and S. Kumar, "Are transformers universal approximators of sequence-to-sequence functions?," in *Proc. Int. Conf. Learn. Represent.*, 2019, pp. 1–23. [Online]. Available: <https://openreview.net/forum?id=ByxRM0Ntvr>
- [32] R. Tutunov, A. Grosnit, J. Ziomek, J. Wang, and H. Bou-Ammar, "Why can large language models generate correct chain-of-thoughts?," 2023, *arXiv:2310.13571*.
- [33] H. Jiang, "A latent space theory for emergent abilities in large language models," 2023, *arXiv:2304.09960*.
- [34] M. Xu et al., "Cached model-as-a-resource: Provisioning large language model agents for edge intelligence in space-air-ground integrated networks," 2024, *arXiv:2403.05826*.
- [35] R. Girdhar et al., "ImageBind one embedding space to bind them all," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2023, pp. 15180–15190.
- [36] S. Roy and D. Roth, "Solving general arithmetic word problems," 2016, *arXiv:1608.01413*.
- [37] P. Clark et al., "Think you have solved question answering? try ARC, the A12 reasoning challenge," 2018, *arXiv:1803.05457*.
- [38] M. Geva, D. Khashabi, E. Segal, T. Khot, D. Roth, and J. Berant, "Did aristotle use a laptop? A question answering benchmark with implicit reasoning strategies," *Trans. Assoc. Comput. Linguistics*, vol. 9, pp. 346–361, Apr. 2021.
- [39] A. Talmor, J. Herzig, N. Lourie, and J. Berant, "CommonsenseQA: A question answering challenge targeting commonsense knowledge," 2018, *arXiv:1811.00937*.
- [40] B. Wang et al., "Adversarial GLUE: A multi-task benchmark for robustness evaluation of language models," 2021, *arXiv:2111.02840*.
- [41] Y. Zhang, Y. Wang, L. Zhang, Z. Zhang, and K. Gai, "Improve diverse text generation by self labeling conditional variational auto encoder," in *Proc. IEEE Int. Conf. Acoust., Speech Signal Process. (ICASSP)*, May 2019, pp. 2767–2771.
- [42] K. Cobbe et al., "Training verifiers to solve math word problems," 2021, *arXiv:2110.14168*.



Minrui Xu (Member, IEEE) received the B.S. degree from Sun Yat-sen University, Guangzhou, China, in 2021. He is currently pursuing the Ph.D. degree with the School of Computer Science and Engineering, Nanyang Technological University, Singapore. His main research interests include large language models over networks, metaverse, deep reinforcement learning, and mechanism design. He was a recipient of the IEEE VTS Daniel E. Noble Fellowship and the 2025 IEEE ICC Best Paper Award.



Dusit Niyato (Fellow, IEEE) received the B.Eng. degree from the King Mongkut's Institute of Technology Ladkrabang (KMUTL), Thailand, and the Ph.D. degree in electrical and computer engineering from the University of Manitoba, Canada. He is currently a Professor with the College of Computing and Data Science, Nanyang Technological University, Singapore. His research interests include mobile generative AI, edge general intelligence, quantum computing and networking, and incentive mechanism design.



Christopher G. Brinton (Senior Member, IEEE) received the M.S. and Ph.D. (Hons.) degrees in electrical engineering from Princeton University in 2016 and 2013, respectively. He is currently an Elmore Associate Professor of electrical and computer engineering (ECE) and the Faculty Director with the HIVE Engineering Entrepreneurship Center, Purdue University. Prior to joining Purdue University, he was the Associate Director of the EDGE Laboratory and a Lecturer of electrical engineering with Princeton University. He also co-founded Zoomi Inc., a big data startup company that has provided learning optimization to more than one million users worldwide. He holds U.S. patents in machine learning for education. He is the author of the book *The Power of Networks: Six Principles That Connect Our Lives* and associated with the Massive Open Online Courses (MOOCs) have reached over 400 000 students. His research interests include the intersection of machine learning, communications, and networking, specifically in fog/edge network intelligence, distributed machine learning, and AI/ML-inspired wireless network optimization. He was a recipient of five U.S. top early career awards from the National Science Foundation (CAREER), the Office of Naval Research (YIP), the Defense Advanced Research Projects Agency (YFA and Director's Fellowship), and the Air Force Office of Scientific Research (YIP). He was also a recipient of the IEEE Communication Society William Bennett Prize Best Paper Award, the Intel Rising Star Faculty Award, the Qualcomm Faculty Award, and the Purdue College of Engineering Faculty Excellence Awards in Early Career Research, Early Career Teaching, and Online Learning. He serves as an Associate Editor for IEEE/ACM TRANSACTIONS ON NETWORKING. Previously, he was an Associate Editor of IEEE TRANSACTIONS ON WIRELESS COMMUNICATIONS.